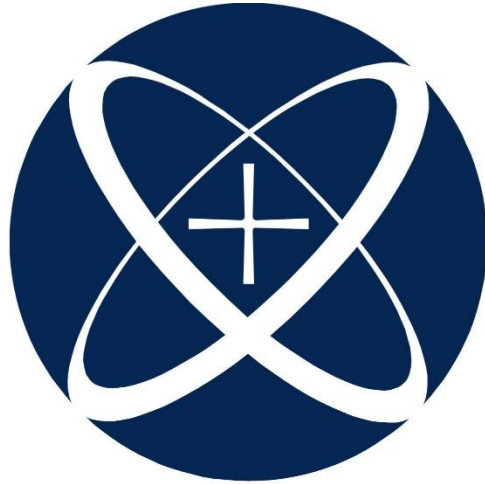




ITESO

Universidad Jesuita
de Guadalajara

Práctica 4

Aplicando algoritmos de búsqueda de caminos

Minería de Grafos

LAMBDA REMIEL HEREDIA PEREZ
4-5-2026

1. Introducción

Los grafos son estructuras matemáticas compuestas por nodos (vértices) y aristas (edges) que permiten modelar relaciones entre entidades del mundo real. En el contexto de la minería de datos, los grafos han cobrado especial relevancia para analizar redes complejas como redes sociales, infraestructuras de transporte, redes biológicas y sistemas de recomendación (Needham & Hodler, 2019).

La búsqueda de caminos (pathfinding) es una de las familias de algoritmos más importantes en la minería de grafos. Estos algoritmos permiten encontrar rutas óptimas entre nodos considerando diferentes criterios de optimización, como la distancia geodésica, el costo o el número de saltos. Entre los algoritmos más utilizados se encuentran el algoritmo de Dijkstra para encontrar el camino más corto en grafos con pesos positivos, el algoritmo de Prim para construir un árbol de expansión mínimo, y los algoritmos de búsqueda en amplitud (BFS) y profundidad (DFS) para exploración de grafos (Scifo, 2023).

Además de los algoritmos de búsqueda de caminos, las métricas de centralidad juegan un papel fundamental en el análisis de grafos. La centralidad de intermediación (Betweenness Centrality) identifica los nodos que actúan como puentes entre distintas partes del grafo; la centralidad de cercanía (Closeness Centrality) mide qué tan rápido un nodo puede alcanzar a todos los demás; el PageRank evalúa la importancia de un nodo en función de los nodos que lo apuntan; y el Degree (grado) contabiliza el número de conexiones de cada nodo (Neo4j Graph Data Science, 2024).

En esta práctica se aplican estos algoritmos y métricas sobre una base de datos de aeropuertos internacionales almacenada en Neo4j, una base de datos orientada a grafos. El grafo contiene 3,214 nodos de tipo Airport y 36,907 relaciones de tipo TO, donde cada relación almacena la distancia geodésica en kilómetros entre aeropuertos. El objetivo es encontrar rutas óptimas y analizar la conectividad de la red aeroportuaria mundial.

2. Importación y Modelado de la Base de Datos

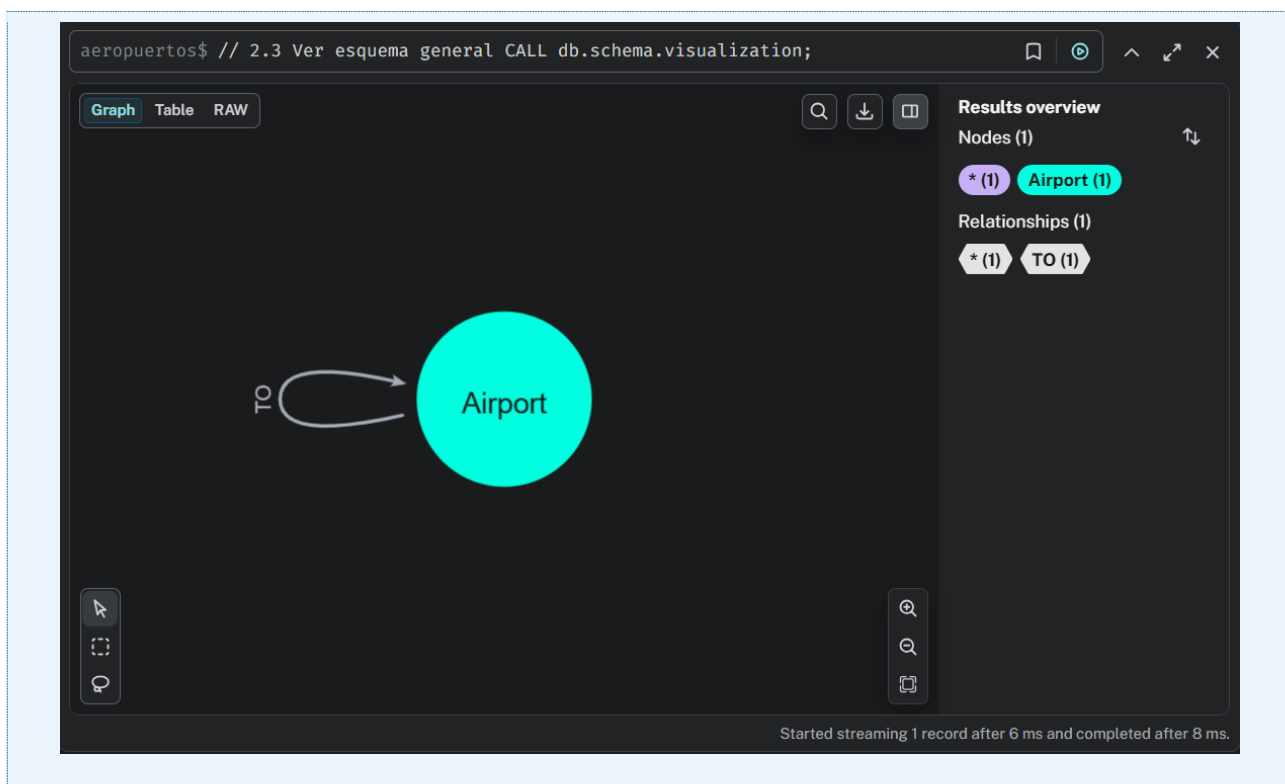
2.1 Proceso de Migración

La base de datos fue restaurada utilizando el archivo aeropuertosdistancia-2026-04-22T17-27-48.dump mediante la sobreescritura de una base de datos, utilizando el menú de la instancia en Neo4j. Una vez cargado el dump, la base de datos fue activada desde la interfaz de Neo4j Desktop bajo el nombre 'aeropuertos'.

2.2 Esquema del Grafo

El esquema del grafo fue verificado mediante el comando `CALL db.schema.visualization`, el cual confirma un único tipo de nodo (Airport) y un único tipo de relación dirigida (TO). Las propiedades del nodo Airport son: Id, Label, country, latitude y longitude. La relación TO contiene la propiedad distance con la distancia en kilómetros.

```
CALL db.schema.visualization
```



3. Análisis Básico del Grafo

Para comprender la estructura general de la red aeroportuaria, se realizaron consultas básicas que permiten conocer el tamaño del grafo, su distribución geográfica y las características de conectividad de sus nodos.

3.1 Conteo de Nodos y Relaciones

```
MATCH (a:Airport) RETURN count(a) AS total_aeropuertos;  
MATCH ()-[r:TO]->() RETURN count(r) AS total_rutas;
```

The screenshot displays two Cypher queries in a Neo4j interface. The first query, executed by user 'aeropuertos\$', counts the relationships of type 'TO' and returns the result '36907' under the alias 'relaciones'. The second query, also by 'aeropuertos\$', counts the nodes of type 'Airport' and returns the result '3214' under the alias 'nodos'. Both queries are shown in a dark-themed editor with a table view of the results.

```
aeropuertos$ MATCH ()-[r:TO]->() RETURN count(r) AS relaciones;  
aeropuertos$ // 3.1 Conteo de nodos y relaciones MATCH (a:Airport) RETURN count(a) AS no
```

relaciones
36907

nodos
3214

3.2 Distribución por País (Top 20)

```
MATCH (a:Airport)  
RETURN a.country AS pais, count(a) AS cantidad  
ORDER BY cantidad DESC  
LIMIT 20;
```

aeropuertos\$ // 3.2 Distribución de aeropuertos por país (Top 20) MATCH (a:Airport) RETU

Table RAW

	pais	cantidad
1	"United States"	549
2	"Canada"	206
3	"China"	173
4	"Brazil"	122
5	"Australia"	113
6	"Russia"	103
7	"India"	68
8	"Indonesia"	63
9	"Japan"	62
10	"Mexico"	56

Started streaming 20 records after 104 ms and completed after 170 ms.

3.3 Aeropuertos con Mayor Grado Total

```
MATCH (a:Airport)
OPTIONAL MATCH (a)-[out:TO]->()
OPTIONAL MATCH ()-[in:TO]->(a)
RETURN a.Label AS aeropuerto, a.country AS pais,
       count(DISTINCT out) AS out_degree,
       count(DISTINCT in) AS in_degree,
       count(DISTINCT out) + count(DISTINCT in) AS total_degree
ORDER BY total_degree DESC
LIMIT 10;
```

aeropuertos\$ // 3.4 Grado promedio (OutDegree) MATCH (a:Airport)-[r:TO]->() RETURN a.Id, a.Label, out_degree

Table RAW

🔍

{ }

⬇

a.Id	a.Label	out_degree
1 340	"Frankfurt"	239
2 1382	"Paris"	237
3 580	"Amsterdam"	232
4 1701	"Istanbul"	224
5 3682	"Atlanta"	217
6 3830	"Chicago"	206
7 3364	"Beijing"	204
8 346	"Munich"	191
9 4029	"Moscow"	187
10 3670	"Dallas-Fort Worth"	187

Started streaming 20 records after 124 ms and completed after 387 ms.

3.4 Estadísticas de Distancia

```
MATCH ()-[r:TO]->()
RETURN min(r.distance) AS distancia_minima_km,
       max(r.distance) AS distancia_maxima_km,
       avg(r.distance) AS distancia_promedio_km;
```

aeropuertos\$ // 3.5 Estadísticas de distancia MATCH ()-[r:TO]->() RETURN min(r.distance)

Table RAW

🔍

{ }

⬇

distancia_minima_km	distancia_maxima_km	distancia_promedio_km
1 0.0	16100.27858603213	1762.153383816451

Started streaming 1 record after 118 ms and completed after 304 ms.

4. Ruta más Corta desde el Aeropuerto de México con Mayor Betweenness Centrality

La Betweenness Centrality mide cuántas veces un nodo actúa como puente en los caminos más cortos entre todos los pares de nodos del grafo. El aeropuerto mexicano con mayor valor de esta métrica es el que mayor influencia tiene sobre el flujo de información o rutas en la red.

4.1 Cálculo de Betweenness Centrality — México

```
CALL gds.betweenness.stream('aeropuertos_dirigido')
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS airport, score
WHERE airport.country = 'Mexico'
RETURN airport.Id AS id, airport.Label AS aeropuerto, score AS betweenness
ORDER BY score DESC
LIMIT 5;
```

aeropuertos\$ // 4.1 Calcular Betweenness Centrality y encontrar el top de México CALL gd

id	aeropuerto	pais	betweenness
1 1824	"Mexico City"	"Mexico"	155323.3852328661
2 1852	"Cancun"	"Mexico"	42113.92261729184
3 1847	"Tijuana"	"Mexico"	8452.466890069947
4 1804	"Guadalajara"	"Mexico"	6879.441698249118
5 1825	"Monterrey"	"Mexico"	2517.004432892255

Started streaming 5 records after 148 ms and completed after 2,472 ms.

4.2 Distancia más Corta a Todos los Nodos (Dijkstra SSSP)

Usando el aeropuerto identificado como origen (mayor Betweenness), se aplica el algoritmo de Dijkstra de fuente única para obtener la distancia mínima hacia todos los demás nodos del grafo dirigido.

```
MATCH (source:Airport {Id: 1824})
CALL gds.allShortestPaths.dijkstra.stream(
  'aeropuertos_dirigido',
  { sourceNode: source, relationshipWeightProperty: 'distance' }
)
```

```

YIELD sourceNode, targetNode, totalCost
RETURN gds.util.asNode(sourceNode).Label AS origen,
       gds.util.asNode(targetNode).Label AS destino,
       gds.util.asNode(targetNode).country AS pais_destino,
       round(totalCost) AS distancia_total_km
ORDER BY totalCost
LIMIT 50;

```

aeropuertos\$ // 4.2 Dijkstra SSSP desde el aeropuerto de México con mayor Betweenness Ce

Table RAW

	origen	destino	pais_destino	distancia_total_km
1	"Mexico City"	"Mexico City"	"Mexico"	0.0
2	"Mexico City"	"Morelia"	"Mexico"	210.0
3	"Mexico City"	"Poza Rico"	"Mexico"	213.0
4	"Mexico City"	"Jalapa"	"Mexico"	239.0
5	"Mexico City"	"Vera Cruz"	"Mexico"	305.0
6	"Mexico City"	"Del Bajio"	"Mexico"	306.0
7	"Mexico City"	"Acapulco"	"Mexico"	307.0
8	"Mexico City"	"Zihuatanejo"	"Mexico"	324.0
9	"Mexico City"	"Tampico"	"Mexico"	342.0
10	"Mexico City"	"Oaxaca"	"Mexico"	368.0

Started streaming 50 records after 199 ms and completed after 2,932 ms.

5. Ruta más Corta: Mayor Closeness Centrality (Italia) → Mayor Degree (Canadá)

La Closeness Centrality mide qué tan cerca está un nodo del resto de nodos en el grafo; un valor alto indica que el aeropuerto puede alcanzar rápidamente a todos los demás. El Degree es el número total de conexiones (entrantes y salientes) de un nodo.

5.1 Aeropuerto de Mayor Closeness Centrality en Italia

```
CALL gds.closeness.stream('aeropuertos_dirigido')
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS airport, score
WHERE airport.country = 'Italy'
RETURN airport.Id AS id, airport.Label AS aeropuerto, score AS closeness
ORDER BY score DESC
LIMIT 5;
```



The screenshot shows a query interface with the command: `aeropuertos$ // 5.1 Calcular Closeness Centrality para Italia CALL gds.closeness.stream()`. The results are displayed in a table with columns `id`, `aeropuerto`, and `closeness`. The table lists the top 5 airports in Italy based on their Closeness Centrality score.

	id	aeropuerto	closeness
1	1555	"Rome"	0.38451268357810414
2	1524	"Milano"	0.360450563204005
3	1551	"Venice"	0.344460150048929
4	1538	"Bologna"	0.32012934518997577
5	1562	"Pisa"	0.315129811996419

Started streaming 5 records after 99 ms and completed after 769 ms.

5.2 Aeropuerto de Mayor Degree en Canadá

```
MATCH (a:Airport {country: 'Canada'})
OPTIONAL MATCH (a)-[out:TO]->()
OPTIONAL MATCH ()-[in:TO]->(a)
RETURN a.Id AS id, a.Label AS aeropuerto,
       count(DISTINCT out) + count(DISTINCT in) AS total_degree
ORDER BY total_degree DESC
LIMIT 5;
```

aeropuertos\$ // 5.2 Encontrar aeropuerto de mayor Degree en Canadá MATCH (a:Airport {cou

id	aeropuerto	total_degree
1 193	"Toronto"	293
2 146	"Montreal"	185
3 156	"Vancouver"	150
4 178	"Calgary"	119
5 49	"Edmonton"	75

Started streaming 5 records after 179 ms and completed after 278 ms.

5.3 Ruta más Corta entre Ambos Aeropuertos

```
// 5.3 Ruta más corta
MATCH (origen:Airport {Id: 1555})
MATCH (destino:Airport {Id: 193})
CALL gds.shortestPath.dijkstra.stream(
  'aeropuertos_dirigido',
  {
    sourceNode: origen,
    targetNode: destino,
    relationshipWeightProperty: 'distance'
  }
)
YIELD nodeId, costs, totalCost
RETURN
[nodeId IN nodeIdIds | gds.util.asNode(nodeId).Label] AS ruta,
[nodeId IN nodeIdIds | gds.util.asNode(nodeId).country] AS paises,
costs AS costos_acumulados_km,
round(totalCost) AS distancia_total_km;

// 5.4 Mapa
MATCH (origen:Airport {Id: 1555})
MATCH (destino:Airport {Id: 193})
CALL gds.shortestPath.dijkstra.stream(
  'aeropuertos_dirigido',
  {
    sourceNode: origen,
    targetNode: destino,
    relationshipWeightProperty: 'distance'
  }
)
YIELD nodeId, costs, totalCost
UNWIND range(0, size(nodeIds)-1) AS i
```

```

WITH gds.util.asNode(nodeIds[i]) AS airport, costs[i] AS costo_acumulado, totalCost
RETURN
  airport,
  round(costo_acumulado) AS km_acumulados,
  round(totalCost)      AS distancia_total_km
ORDER BY km_acumulados;

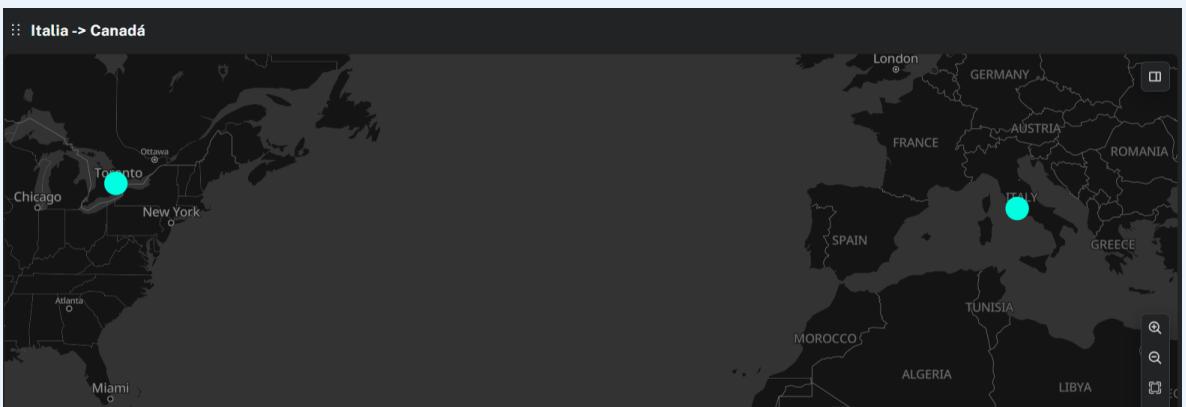
```

aeropuertos\$ // 5.3 Ruta más corta MATCH (origen:Airport {Id: 1555}) MATCH (destino:Airp

Table RAW

ruta	países	costos_acumulados_km	dista
["Rome", "Toronto"]	["Italy", "Canada"]	[0.0, 7093.571978242643]	709 4.0

Started streaming 1 record after 8 ms and completed after 130 ms.

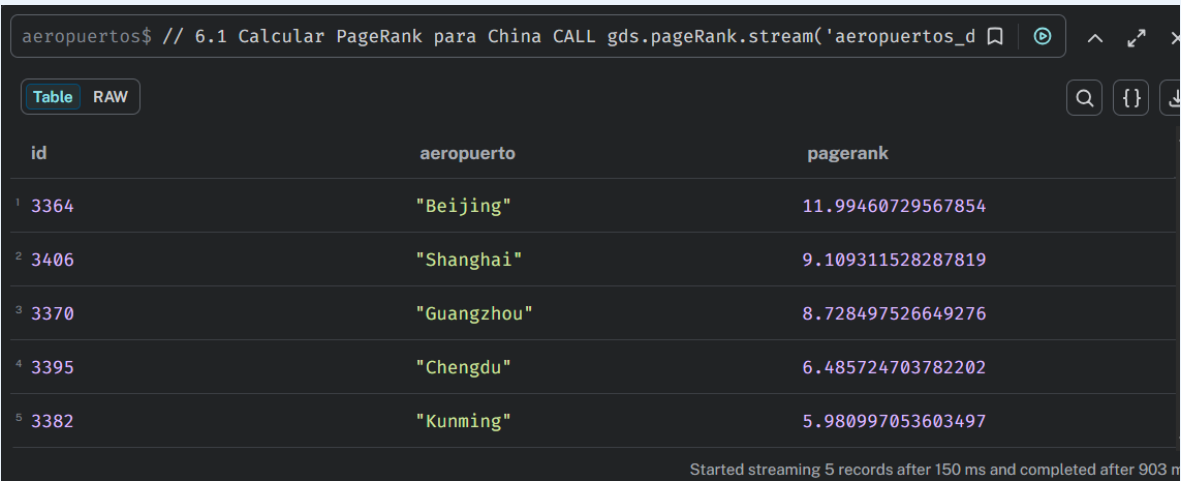


6. Ruta con Detalle de Costo: Mayor PageRank (China) → Mayor Degree (EE.UU.)

El PageRank asigna importancia a un nodo en función de la cantidad e importancia de los nodos que lo enlazan. Un aeropuerto con alto PageRank es un hub estratégico en la red. Se obtiene la ruta hacia el aeropuerto estadounidense con mayor grado total, mostrando el costo acumulado en cada nodo intermedio.

6.1 Aeropuerto de Mayor PageRank en China

```
CALL gds.pageRank.stream('aeropuertos_dirigido')
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS airport, score
WHERE airport.country = 'China'
RETURN airport.Id AS id, airport.Label AS aeropuerto, score AS pagerank
ORDER BY score DESC
LIMIT 5;
```



aeropuertos\$ // 6.1 Calcular PageRank para China CALL gds.pageRank.stream('aeropuertos_d

	id	aeropuerto	pagerank
1	3364	"Beijing"	11.99460729567854
2	3406	"Shanghai"	9.109311528287819
3	3370	"Guangzhou"	8.728497526649276
4	3395	"Chengdu"	6.485724703782202
5	3382	"Kunming"	5.980997053603497

Started streaming 5 records after 150 ms and completed after 903 ms

6.2 Aeropuerto de Mayor Degree en Estados Unidos

```
MATCH (a:Airport {country: 'United States'})
OPTIONAL MATCH (a)-[out:TO]->()
OPTIONAL MATCH ()-[in:TO]->(a)
RETURN a.Id AS id, a.Label AS aeropuerto,
       count(DISTINCT out) + count(DISTINCT in) AS total_degree
ORDER BY total_degree DESC
LIMIT 5;
```

aeropuertos\$ // 6.2 Aeropuerto de mayor Degree en Estados Unidos MATCH (a:Airport {count

id	aeropuerto	total_degree
1 3682	"Atlanta"	433
2 3830	"Chicago"	409
3 3670	"Dallas-Fort Worth"	372
4 3550	"Houston"	337
5 3751	"Denver"	335

Started streaming 5 records after 116 ms and completed after 476 ms.

6.3 Ruta con Costo Acumulado por Nodo

```

MATCH (origen:Airport {Id: 3364}), (destino:Airport {Id: 3682}) // Sustituir Ids reales
CALL gds.shortestPath.dijkstra.stream(
  'aeropuertos_dirigido',
  { sourceNode: origen, targetNode: destino, relationshipWeightProperty:
'distance' }
)
YIELD nodeIds, costs, totalCost
UNWIND range(0, size(nodeIds)-1) AS i
RETURN gds.util.asNode(nodeIds[0]).Label AS aeropuerto_origen,
       gds.util.asNode(nodeIds[i]).Id AS id_nodo,
       gds.util.asNode(nodeIds[i]).Label AS aeropuerto,
       gds.util.asNode(nodeIds[i]).country AS pais,
       round(costs[i]) AS costo_acumulado_km,
       round(totalCost) AS costo_total_km;

```

aeropuertos\$ // 6.3 Ruta con detalle de costo acumulado por nodo MATCH (origen:Airport {

aeropuerto_origen	id_nodo	aeropuerto	pais	costo_acumulado_k	costo_total_km
1 "Beijing"	3364	"Beijing"	"China"	0.0	11552.0
2 "Beijing"	3830	"Chicago"	"United States"	10574.0	11552.0
3 "Beijing"	3682	"Atlanta"	"United States"	11552.0	11552.0

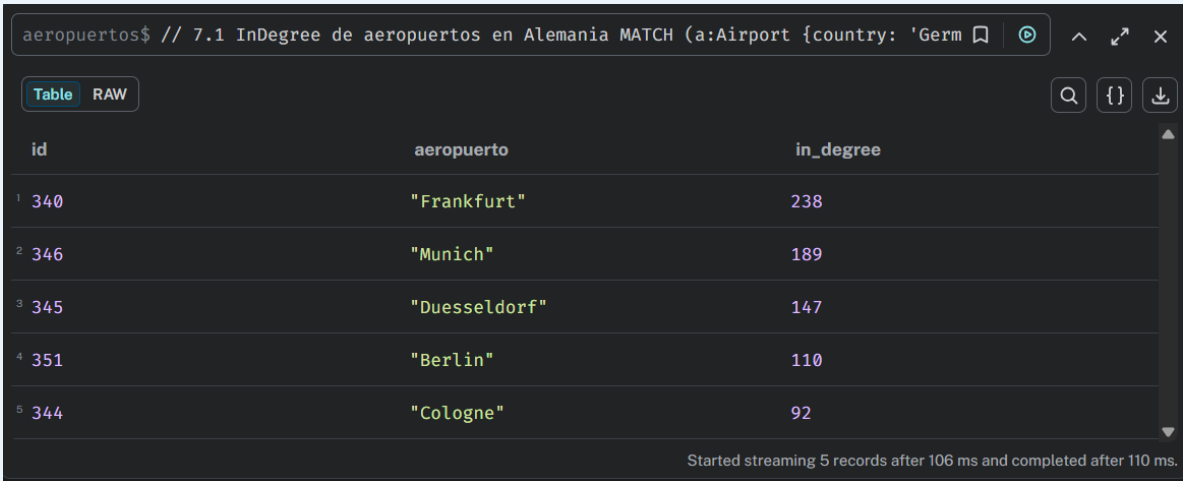
Started streaming 3 records after 147 ms and completed after 165 ms.

7. Ruta en Bloom: Mayor InDegree (Alemania) → Mayor InDegree (Grecia)

El InDegree cuenta únicamente las relaciones entrantes de un nodo. Un aeropuerto con alto InDegree es un destino muy popular en la red. Se visualiza la ruta completa usando Neo4j Bloom (Explore).

7.1 Aeropuerto de Mayor InDegree en Alemania

```
MATCH (a:Airport {country: 'Germany'})
OPTIONAL MATCH ()-[in:TO]->(a)
RETURN a.Id AS id, a.Label AS aeropuerto, count(in) AS in_degree
ORDER BY in_degree DESC
LIMIT 5;
```



The screenshot shows the Neo4j Bloom interface with a query executed. The query is: `aeropuertos$ // 7.1 InDegree de aeropuertos en Alemania MATCH (a:Airport {country: 'Germ`. The results are displayed in a table with columns: `id`, `aeropuerto`, and `in_degree`. The table shows the top 5 airports in Germany by in-degree.

	id	aeropuerto	in_degree
1	340	"Frankfurt"	238
2	346	"Munich"	189
3	345	"Duesseldorf"	147
4	351	"Berlin"	110
5	344	"Cologne"	92

Started streaming 5 records after 106 ms and completed after 110 ms.

7.2 Aeropuerto de Mayor InDegree en Grecia

```
MATCH (a:Airport {country: 'Greece'})
OPTIONAL MATCH ()-[in:TO]->(a)
RETURN a.Id AS id, a.Label AS aeropuerto, count(in) AS in_degree
ORDER BY in_degree DESC
LIMIT 5;
```

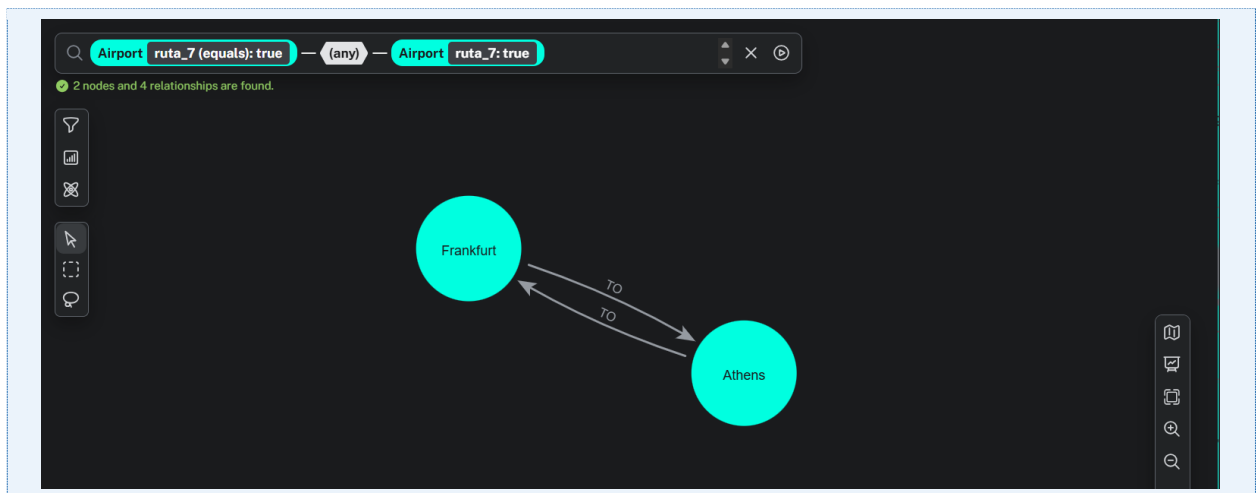
aeropuertos\$ // 7.2 InDegree de aeropuertos en Grecia MATCH (a:Airport {country: 'Greece

id	aeropuerto	in_degree
1 3941	"Athens"	102
2 1452	"Heraklion"	58
3 1486	"Thessaloniki"	55
4 1472	"Rhodos"	50
5 1474	"Chania"	38

Started streaming 5 records after 36 ms and completed after 38 ms.

7.3 Visualización de la Ruta en Neo4j Bloom

La ruta completa desde el aeropuerto alemán hasta el griego fue visualizada usando Neo4j Bloom (Explore), mostrando todos los nodos y aristas que conforman el camino más corto.

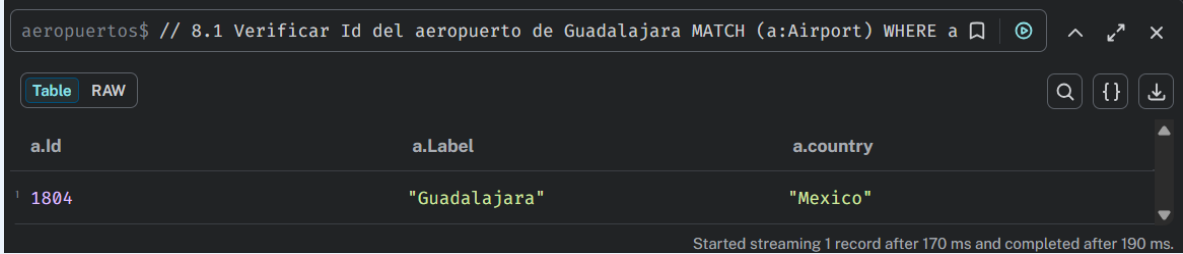


8. Árbol de Expansión Mínimo (MST) desde el Aeropuerto de Guadalajara

El árbol de expansión mínimo (Minimum Spanning Tree) encuentra la forma más económica de conectar todos los nodos alcanzables desde un origen, minimizando la distancia total recorrida. Se utiliza el algoritmo de Prim implementado en la librería GDS de Neo4j.

8.1 MST desde Guadalajara (GDL)

```
MATCH (source:Airport {Id: 1804})
CALL gds.spanningTree.stream(
  'aeropuertos_nodirigido',
  { sourceNode: source, relationshipWeightProperty: 'distance', objective:
'minimum' }
)
YIELD nodeId, parentId, weight
RETURN gds.util.asNode(nodeId).Label AS aeropuerto,
       gds.util.asNode(nodeId).country AS pais,
       gds.util.asNode(parentId).Label AS conectado_desde,
       round(weight) AS distancia_km
ORDER BY weight;
```



a.id	a.Label	a.country
1804	Guadalajara	Mexico

8.2 Costo Total del MST

```
MATCH (source:Airport {Id: 1804})
CALL gds.spanningTree.stream(
  'aeropuertos_nodirigido',
  { sourceNode: source, relationshipWeightProperty: 'distance', objective:
'minimum' }
)
YIELD nodeId, weight
RETURN count(nodeId) AS nodos_conectados, round(sum(weight)) AS costo_total_km;
```

aeropuertos\$ // 8.3 Costo total del MST desde Guadalajara MATCH (source:Airport {Id: 180

TableRAW

🔍

{ }

⬇

nodos_conectados	costo_total_km
3188	1233249.0

Started streaming 1 record after 86 ms and completed after 201 ms.

9. Aeropuertos Fuera del Alcance Mínimo desde el Mayor PageRank de Francia

Se identifica el aeropuerto francés con mayor PageRank y se calculan todos los nodos alcanzables desde él mediante BFS. Los aeropuertos que no aparecen en el resultado son los que quedan fuera de esa ruta de alcance mínimo.

9.1 Mayor PageRank en Francia

```
CALL gds.pageRank.stream('aeropuertos_dirigido')
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS airport, score
WHERE airport.country = 'France'
RETURN airport.Id AS id, airport.Label AS aeropuerto, score AS pagerank
ORDER BY score DESC
LIMIT 5;
```

	id	aeropuerto	pagerank
1	1382	"Paris"	12.528642615758152
2	1386	"Paris"	7.1413899992141125
3	1353	"Marseille"	4.229816452700605
4	1354	"Nice"	4.0804918627694
5	1335	"Lyon"	3.9750851652705026

Started streaming 5 records after 13 ms and completed after 97 ms.

9.2 Aeropuertos NO Alcanzables desde el Nodo Origen

```
MATCH (source:Airport {Id: 1382})
MATCH (target:Airport)
WHERE target <> source
AND NOT EXISTS {
  MATCH (source)-[:TO*]->(target)
}
RETURN target.Id AS id, target.Label AS aeropuerto, target.country AS pais
ORDER BY pais, aeropuerto;
```

aeropuertos\$ // 9.3 Aeropuertos que quedan FUERA de la ruta alcanzable MATCH (source:Air

Table RAW

Q {}

id	aeropuerto	pais
¹ 3334	"Melbourne"	"Australia"
² 6321	"Portland"	"Australia"
³ 7368	"Gurupi"	"Brazil"
⁴ 7375	"Minacu"	"Brazil"
⁵ 7374	"Santa Terezinha"	"Brazil"
⁶ 8243	"Sao Felix do Araguaia"	"Brazil"
⁷ 8241	"Sao Felix do Xingu"	"Brazil"
⁸ 5500	"Vancouver"	"Canada"
⁹ 4106	"Victoria"	"Canada"
¹⁰ 5653	"Lodja"	"Congo (Kinshasa)"

Started streaming 48 records after 162 ms and completed after 280 ms.

10. Aeropuertos Alcanzables desde Cancún sin Superar 10,000 km

Usando el algoritmo de Dijkstra de fuente única (SSSP) desde el Aeropuerto Internacional de Cancún (CUN), se filtran únicamente los destinos cuya distancia acumulada no supere los 10,000 kilómetros, sin repetir aeropuertos en la ruta.

10.1 Aeropuertos Alcanzables ($\leq 10,000$ km)

```
MATCH (source:Airport {Id: 1852})
CALL gds.allShortestPaths.dijkstra.stream(
  'aeropuertos_dirigido',
  { sourceNode: source, relationshipWeightProperty: 'distance' }
)
YIELD targetNode, totalCost
WHERE totalCost <= 10000 AND totalCost > 0
RETURN gds.util.asNode(targetNode).Id AS id,
       gds.util.asNode(targetNode).Label AS aeropuerto,
       gds.util.asNode(targetNode).country AS pais,
       round(totalCost) AS distancia_acumulada_km
ORDER BY totalCost;
```

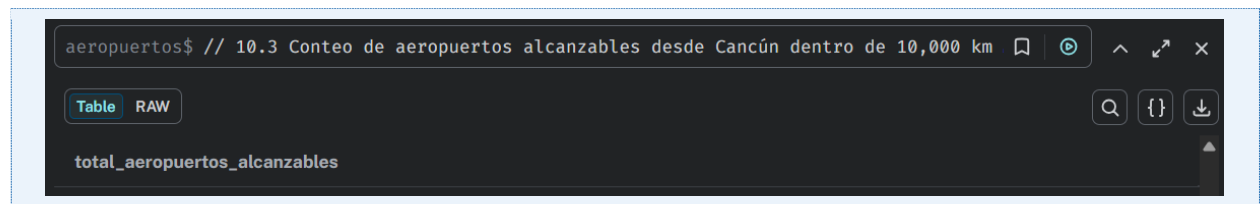
aeropuertos\$ // 10.2 Dijkstra desde Cancún con límite de 10,000 km (Single Source) MATCH

id	aeropuerto	pais	distancia_acumulada_km
1800	"Cozumel"	"Mexico"	57.0
1819	"Merida"	"Mexico"	289.0
1957	"Belize City"	"Belize"	417.0
4279	"San Pedro"	"Belize"	472.0
1909	"Havana"	"Cuba"	510.0
4024	"Flores"	"Guatemala"	597.0
1774	"Roatan"	"Honduras"	651.0
1772	"San Pedro Sula"	"Honduras"	653.0
1903	"Cayo Largo del Sur"	"Cuba"	687.0
1853	"Villahermosa"	"Mexico"	709.0

Started streaming 1,716 records after 116 ms and completed after 270 ms.

10.2 Total de Aeropuertos Alcanzables

```
MATCH (source:Airport {Id: 1852})
CALL gds.allShortestPaths.dijkstra.stream(
  'aeropuertos_dirigido',
  { sourceNode: source, relationshipWeightProperty: 'distance' }
)
YIELD targetNode, totalCost
WHERE totalCost <= 10000 AND totalCost > 0
RETURN count(targetNode) AS total_aeropuertos_alcanzables;
```



aeropuertos\$ // 10.3 Conteo de aeropuertos alcanzables desde Cancún dentro de 10,000 km

Table RAW

total_aeropuertos_alcanzables

11. Ruta dentro del Presupuesto (25,000 pesos / 18 pesos por km)

Dado un costo de 18 pesos por kilómetro y un presupuesto de 25,000 pesos, la distancia máxima permitida es:

$$25,000 \div 18 = 1,388.88 \text{ km máximo}$$

Se busca la ruta más larga posible desde el aeropuerto de origen sin superar este límite, mostrando el detalle del presupuesto gastado en cada escala.

11.1 Aeropuertos Alcanzables dentro del Presupuesto

```
MATCH (source:Airport {Id: 1804})
CALL gds.allShortestPaths.dijkstra.stream(
  'aeropuertos_dirigido',
  { sourceNode: source, relationshipWeightProperty: 'distance' }
)
YIELD targetNode, totalCost, nodeIds
WHERE totalCost <= 1388.88 AND totalCost > 0
RETURN gds.util.asNode(nodeIds[0]).Label AS aeropuerto_origen,
       gds.util.asNode(targetNode).Label AS aeropuerto_destino,
       gds.util.asNode(targetNode).country AS pais_destino,
       round(totalCost) AS distancia_total_km,
       round(totalCost * 18) AS costo_total_pesos,
       25000 - round(totalCost * 18) AS presupuesto_restante_pesos
ORDER BY totalCost DESC;
```

aeropuertos\$ // 11.2 Ruta desde Cancún (o el aeropuerto elegido) dentro del presupuesto

	aeropuerto_o	aeropuerto_c	pais_destino	ruta_completa	distancia_tot	costo_total_	presupuesto.
1	"Guadalajara"	"Campeche"	"Mexico"	["Guadalajara", "Mexico City", "Campeche"]	1359.0	24467.0	533.0
2	"Guadalajara"	"Nuevo Laredo"	"Mexico"	["Guadalajara", "Mexico City", "Nuevo Laredo"]	1352.0	24345.0	655.0
3	"Guadalajara"	"Tapachula"	"Mexico"	["Guadalajara", "Mexico City", "Tapachula"]	1340.0	24120.0	880.0
4	"Guadalajara"	"Houston"	"United States"	["Guadalajara", "Houston"]	1323.0	23817.0	1183.0
5	"Guadalajara"	"Ciudad Juarez"	"Mexico"	["Guadalajara", "Ciudad Juarez"]	1276.0	22963.0	2037.0
6	"Guadalajara"	"Queretaro"	"Mexico"	["Guadalajara", "Monterrey", "Queretaro"]	1245.0	22416.0	2584.0
7	"Guadalajara"	"Hermosillo"	"Mexico"	["Guadalajara", "Hermosillo"]	1233.0	22193.0	2807.0

Started streaming 49 records after 50 ms and completed after 85 ms.

11.2 Detalle Nodo por Nodo de la Ruta Elegida

```
MATCH (origen:Airport {Id: 1804}), (destino:Airport {Id: 1795})
CALL gds.shortestPath.dijkstra.stream(
  'aeropuertos_dirigido',
  {
    sourceNode: origen,
    targetNode: destino,
    relationshipWeightProperty: 'distance'
  }
)
YIELD nodeIds, costs, totalCost
UNWIND range(0, size(nodeIds)-1) AS i
RETURN
  gds.util.asNode(nodeIds[0]).Label AS aeropuerto_origen,
  i AS paso,
  gds.util.asNode(nodeIds[i]).Id AS id_aeropuerto,
  gds.util.asNode(nodeIds[i]).Label AS aeropuerto,
  gds.util.asNode(nodeIds[i]).country AS pais,
  round(costs[i]) AS km_acumulados,
  round(costs[i] * 18) AS pesos_gastados_acumulados,
  round(totalCost) AS km_totales,
  round(totalCost * 18) AS costo_total_pesos,
  25000 - round(totalCost * 18) AS presupuesto_restante;
```

aeropuertos\$ // 11.3 Detalle del presupuesto gastado nodo por nodo en la ruta más lejana

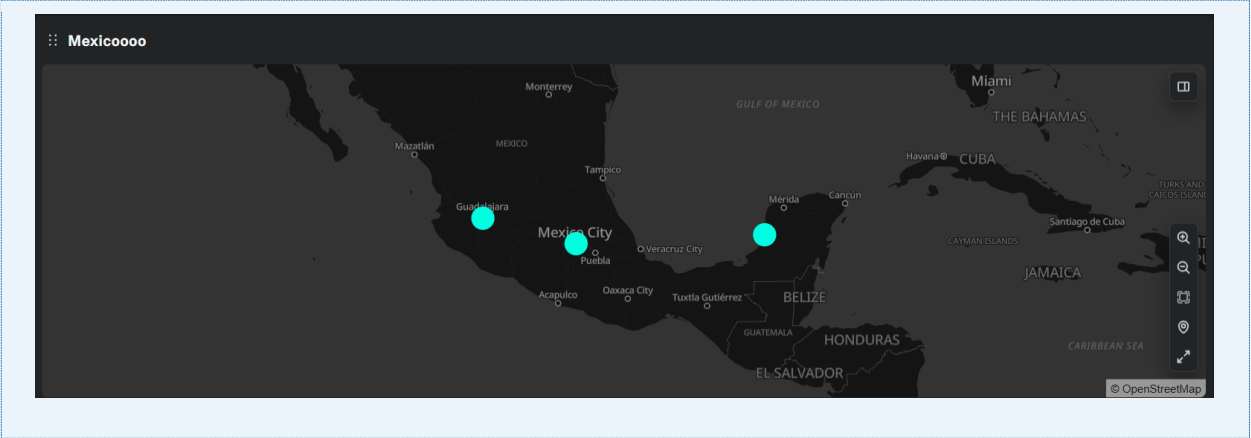
TableRAW

	aeropuerto	paso	id_aeropue	aeropuerto	pais	km_acumu	pesos_gas	km_totales	costo_total	presupuest
1	"Guadala jara"	0	1804	"Guadalaj ara"	"Mexico"	0.0	0.0	1359.0	24467.0	533.0
2	"Guadala jara"	1	1824	"Mexico C ity"	"Mexico"	460.0	8273.0	1359.0	24467.0	533.0
3	"Guadala jara"	2	1795	"Campech e"	"Mexico"	1359.0	24467.0	1359.0	24467.0	533.0

Started streaming 3 records after 171 ms and completed after 180 ms.

> ⓘ 03N90: Cartesian product

11.3 Visualización de la Ruta en Mapa



12. Conclusiones y Recomendaciones

12.1 Hallazgos

A lo largo de esta práctica se pudo comprobar la potencia de los algoritmos de búsqueda de caminos aplicados sobre una red real de aeropuertos internacionales. El uso de métricas de centralidad como Betweenness, Closeness, PageRank y Degree permitió identificar los nodos más estratégicos de cada país analizado, revelando que los principales hubs mundiales coinciden con aeropuertos ampliamente reconocidos por su volumen de tráfico.

El algoritmo de Dijkstra demostró ser altamente eficiente para encontrar rutas óptimas en grafos con pesos positivos, mientras que el Minimum Spanning Tree permitió visualizar la estructura mínima de conexión de la red desde Guadalajara. La consulta de presupuesto evidenció cómo restricciones económicas limitan significativamente el alcance de una ruta, lo que tiene aplicaciones directas en planificación de vuelos y logística.

12.2 Recomendaciones Personales

Se recomienda explorar la integración de datos adicionales como frecuencia de vuelos o capacidad de pasajeros para enriquecer el análisis. Asimismo, el uso de NeoDash para la visualización geoespacial de rutas en mapas resulta una herramienta muy intuitiva que complementa los resultados obtenidos en Neo4j Browser y Bloom. Para proyectos futuros, sería valioso aplicar algoritmos de detección de comunidades para identificar clústeres de aeropuertos con mayor conectividad regional.

Referencias Bibliográficas

- Needham, M., & Hodler, A. E. (2019). Graph algorithms: practical examples in Apache Spark and Neo4j. O'Reilly Media.
- Scifo, E. (2023). Graph Data Science with Neo4j: Learn How to Use Neo4j 5 with Graph Data Science Library 2.0 and Its Python Driver for Your Project. Packt Publishing.
- Neo4j. (2024). Graph Data Science — Centrality Algorithms. Recuperado de <https://neo4j.com/docs/graph-data-science/current/algorithms/centrality/>
- Neo4j. (2024). Cypher Cheat Sheet — Neo4j Enterprise 5. Recuperado de <https://neo4j.com/docs/cypher-cheat-sheet/5/neo4j-enterprise/>
- Neo4j. (2024). Neo4j Bloom User Guide. Recuperado de <https://neo4j.com/docs/bloom-user-guide/current/>